 Nested Loops in List Comprehension (with Examples)

1. Normal Nested Loop

Suppose we want all pairs (x, y) where x is from [1, 2, 3] and y is from [10, 20].

```
pairs = []
for x in [1, 2, 3]:
    for y in [10, 20]:
        pairs.append((x, y))

print(pairs)
```

Output:

```
[(1, 10), (1, 20), (2, 10), (2, 20), (3, 10), (3, 20)]
```

2. Same with List Comprehension

```
pairs = [(x, y) for x in [1, 2, 3] for y in [10, 20]]
print(pairs)
```

Output:

```
[(1, 10), (1, 20), (2, 10), (2, 20), (3, 10), (3, 20)]
```

3. Explanation

The comprehension order is the same as the nested loops:

```
for x in [1, 2, 3]:
    for y in [10, 20]:
        (x, y)
```

becomes:

```
[(x, y) for x in [1, 2, 3] for y in [10, 20]]
```

4. With a Condition

Say we only want pairs where x * y > 20:

```
pairs = [(x, y) for x in [1, 2, 3] for y in [10, 20] if x * y > 20]
print(pairs)
```

Output:

```
[(2, 20), (3, 10), (3, 20)]
```

5. More Complex Example (Flatten a Matrix)

```
matrix = [[1, 2, 3], [4, 5, 6]]
flat = [num for row in matrix for num in row]
print(flat)
```

Output:

```
[1, 2, 3, 4, 5, 6]
```

 Rule of Thumb: • Order of for in list comprehension = order of nested loops in code.
• You can add conditions (if ...) at the end for filtering.